

MediClaim Orchestrator

The Project, Explained Simply

Problem, idea, tools, workflow and settings, in plain English

Multi-Agent Cashless and Reimbursement Claim Processing

Read this first. The second PDF explains every workflow.

October 2026

Contents

1. The whole project in one minute.....	3
2. The problem.....	4
2.1 What happens today.....	4
2.2 Why this is hard.....	4
2.3 Our problem statement.....	4
3. What we built.....	5
3.1 The hospital side.....	5
3.2 The insurer side.....	5
4. Why we did it this way.....	8
5. Why CrewAI, and what was wrong before.....	9
5.1 The cause.....	9
5.2 What CrewAI gives us.....	9
5.3 Our agents.....	10
5.4 The flows.....	11
6. Who does what.....	12
7. The tech stack: every tool and why.....	14
7.1 The AI part.....	14
7.2 The workflow part.....	14
7.3 The backend part.....	15
7.4 The storage part.....	15
7.5 The safety part.....	16
7.6 The document part.....	16
7.7 The screen part.....	17
7.8 The 'make it run' part.....	17
8. Triggers: what starts each workflow.....	18
9. How the decision is made.....	20
9.1 The decision gate.....	20
9.2 The estimate: an example.....	21
10. The parameters: settings you can change.....	22
10.1 Business settings (the rules).....	22
10.2 AI and agent settings.....	22
10.3 Running settings.....	23
10.4 Parameters inside the code.....	23
11. Keeping data safe.....	24
12. How to run it.....	25
12.1 What we got.....	25
13. What is still left, and what comes next.....	26
14. Questions the examiner may ask.....	27

1. The whole project in one minute

When a patient with health insurance is admitted to a hospital, somebody has to get the money from the insurance company. Today this is mostly paper work. The hospital collects bills and reports, types them into a form, and sends them. The insurance company opens every file, checks everything by hand, asks questions, and finally pays.

We built software that does this job for both sides. It reads the documents, builds the claim, checks it, sends it, verifies it on the insurer side, decides, and pays. It uses **AI agents** for the work that needs reading and writing, and **normal code** for anything with rules or money.

Everything runs on one laptop, with a free AI model, and starts with **one command**: ``make demo``.

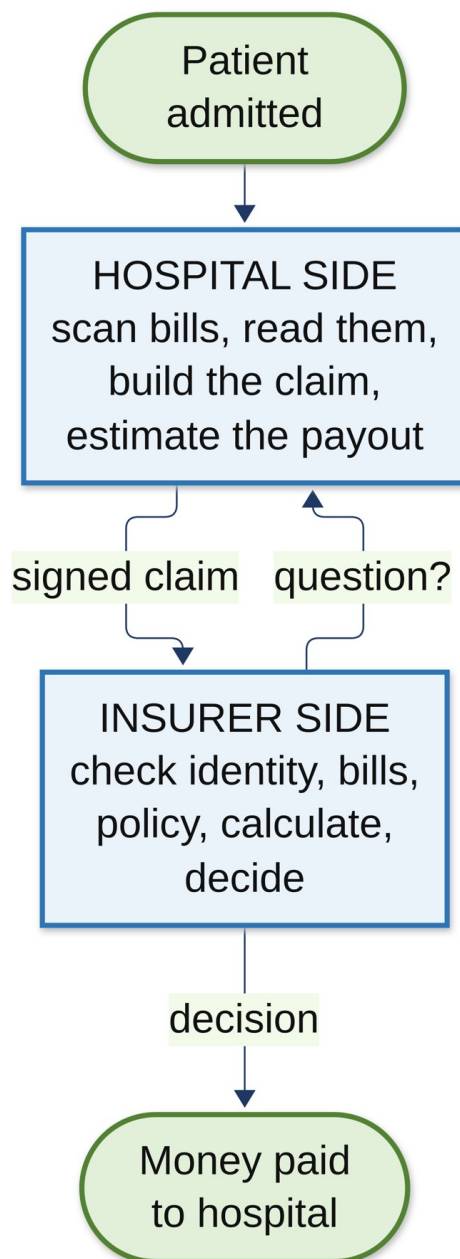


Figure 1: The big picture: two sides, one claim

2. The problem

2.1 What happens today

Let's take an example. Ravi is admitted for an appendix operation. The bill comes to ₹55,800. His policy is cashless, so the hospital should get the money directly from the insurer. Here is what happens:

1. The hospital desk collects the bills, the discharge summary, lab reports, the ID card and the policy card. Many are scanned photos.
2. Someone types the details into the insurer's form. Mistakes happen here: a wrong date, a wrong total, a missing page.
3. The insurer's team opens every page, checks the name, checks the policy, checks the bill maths, checks what the policy covers.
4. If anything is missing, they send a **query** back. The hospital replies. This can go back and forth many times.
5. Finally someone decides how much to pay, and the money is sent.

Ravi waits in the hospital the whole time. Discharge gets delayed by hours.

2.2 Why this is hard

- **Documents are messy.** Scanned, handwritten, different formats from every hospital.
- **Policies are complicated.** Room rent limits, co-pay, sub-limits, things not covered. Even staff get them wrong.
- **Too many queries.** Small mistakes cause back and forth, and each round takes hours or days.
- **Money must be exact.** One wrong rupee is a real problem, and every decision must be explained.
- **Health data is private.** Under India's data protection law (DPDP Act), we cannot just send patient data anywhere.

2.3 Our problem statement

Problem statement

Build a multi-agent system where the hospital side turns raw documents into a correct, checked claim, and the insurer side verifies and decides on it, with AI agents doing the reading and writing, fixed code doing the rules and money, people approving the big or doubtful cases, and everything running locally and starting with one command.

3. What we built

We built **two separate systems**, one for the hospital and one for the insurer, because in real life they are two different companies. They don't share a database. They talk only through a signed API, the same way two banks talk.

3.1 The hospital side

- The desk uploads documents on a web page.
- Each file is virus-scanned and quality-checked (is the photo blurry?).
- The text is read with OCR, and fields like name, dates, amounts are picked out. Personal details are hidden before any AI sees them.
- A checklist shows which documents are still missing.
- **CrewAI agents build the claim:** they sort every bill line into a category, and they **estimate how much the insurer will pay**, by reading the patient's own policy wording.
- 12 checks run on the claim (we call them V01 to V12). The billing officer looks at it, fixes anything, and signs off.
- The claim is sent to the insurer, signed, so nobody can change it on the way.

3.2 The insurer side

- The claim arrives and verification starts automatically.
- **CrewAI agents check** identity (is this the right patient?), authenticity (are the bills genuine?), coverage (what does the policy say?) and map the bill lines for calculation.
- **The calculation engine** works out the exact payable amount. No AI touches the money.
- **The decision gate** decides who approves: the system itself for small clean claims, one person for bigger ones, two people for very big ones.
- If something is missing, an agent drafts a **query** to the hospital. The hospital's agent sorts the query and drafts a reply for the officer.
- Once approved, the money is paid through a bank simulator, and both sides keep an audit trail that cannot be secretly edited.

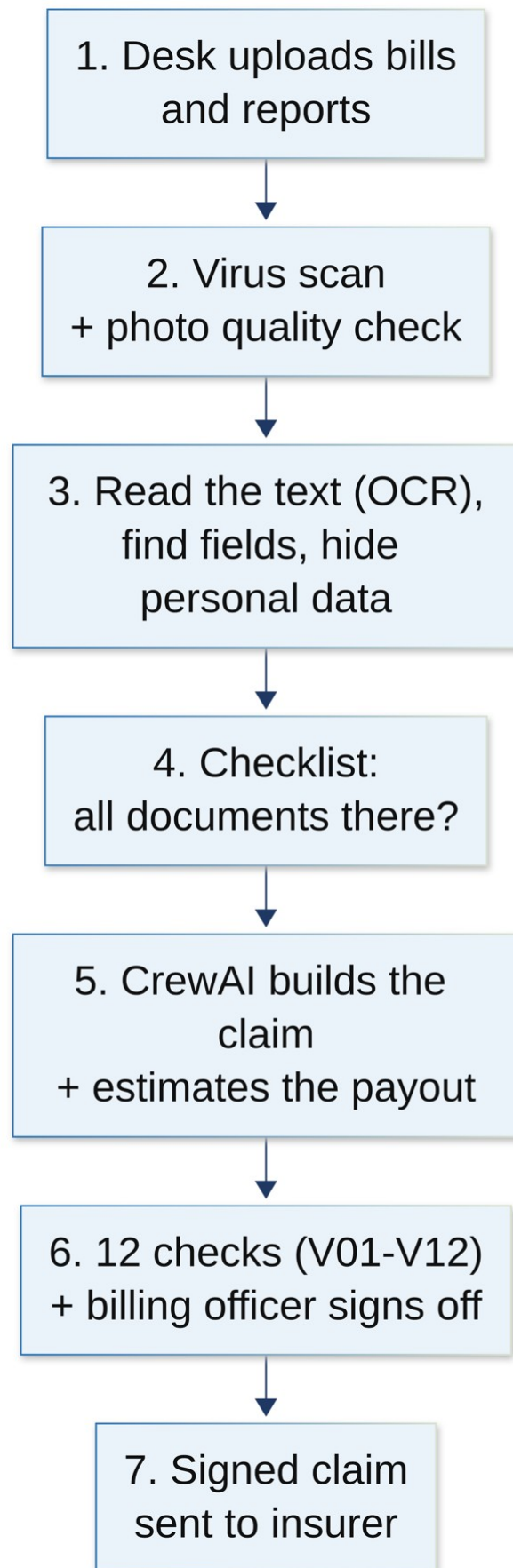


Figure 2: A claim's journey, part 1: the hospital side

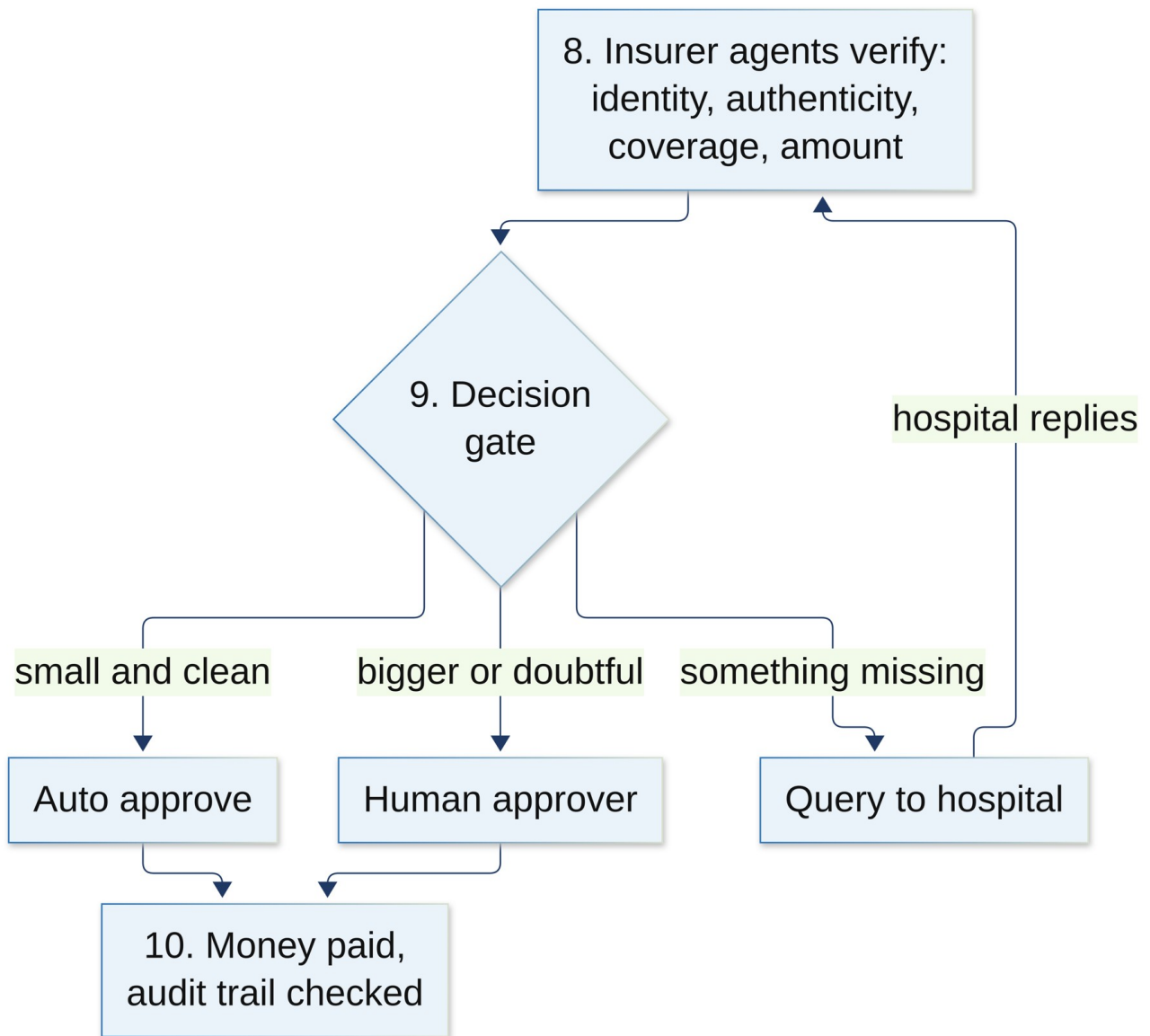


Figure 3: A claim's journey, part 2: the insurer side

4. Why we did it this way

These are the big choices we made, and the reason for each one.

Our choice	Why
AI only where it is really needed	AI is good at reading messy text and writing. It is bad at maths and can make things up. So AI reads and writes, and plain code does checks and money.
Code has the final say	If an agent says 'all fine' but the code finds the name doesn't match, the code wins. The claim goes to a person.
Run the AI on our own laptop	Patient data never leaves the machine. No cost per call. No API key. Works without internet.
Two separate systems	Hospital and insurer are different companies in real life. Keeping them separate is closer to reality and safer.
People approve big claims	The system only auto-approves small and clean claims. Anything big or doubtful goes to a human.
One command to run it all	Anyone (including the examiner) can see it working without a long setup.
Everything is recorded	Every step goes into an audit log that is chained with hashes, so any change can be detected.

5. Why CrewAI, and what was wrong before

5.1 The cause

Our project guidelines ask for a **multi-agent system** built with a proper agent framework. When we went through the earlier code, the 'agents' were just **plain Python files** that called the AI model directly. There was no real agent framework. Every agent was hardcoded, the order of steps was hidden inside the code, and there was nothing checking the agent's answer in a standard way.

That did not meet the guidelines. So we moved the agents to **CrewAI**.

5.2 What CrewAI gives us

CrewAI word	Simple meaning	In our project
Agent	A worker with a role and a goal, like 'claims desk reply writer'	Written in <code>config/agents.yaml</code> on both sides
Task	One job for that worker, like 'map these bill lines'	One task for each AI call
Crew	A team of agents doing tasks together	Runs the task with its agent
Flow	The order of steps: first this, then that, and if X then go here	ClaimFlow, QueryFlow (hospital); VerificationFlow, QueryFlow (insurer)
Guardrail	A checker that looks at the agent's answer. If it's wrong, the agent tries again with feedback	Quote checks, allowed-category checks, grounding rules

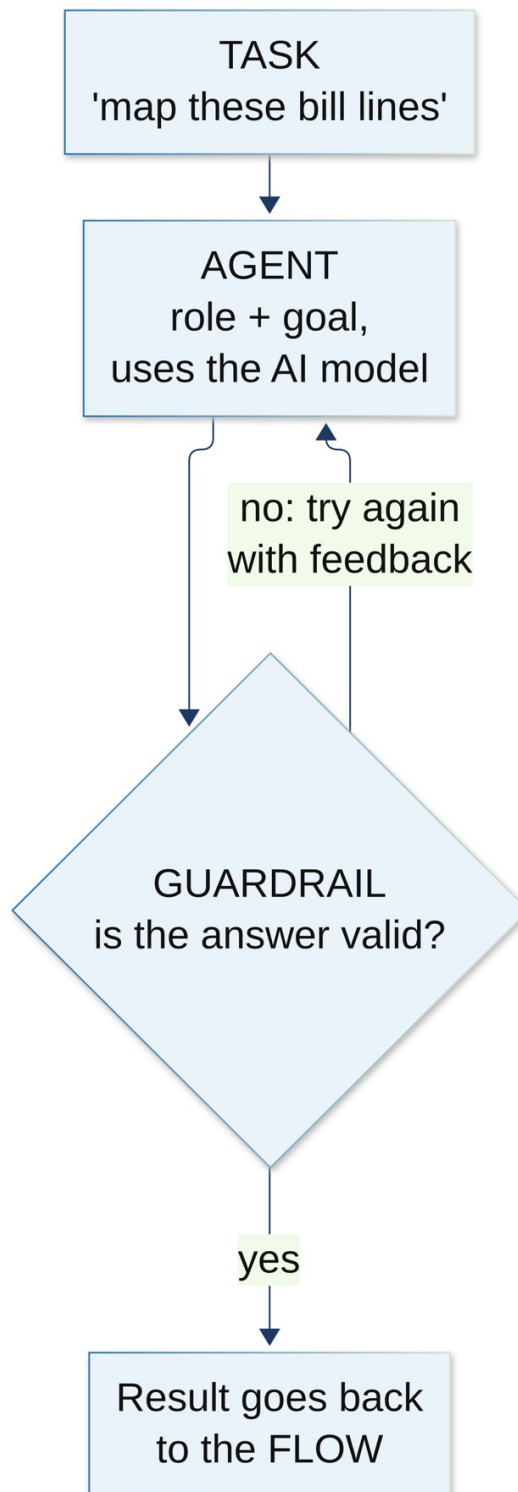


Figure 4: How one CrewAI task works: the agent answers, the guardrail checks, a bad answer goes back

5.3 Our agents

Hospital side (4 agents)

- **Category mapper:** puts each bill line into a group (room, medicine, lab test, surgery...). Only allowed groups are accepted.

- **Policy estimator:** reads the patient's policy wording and finds room rent limit, ICU limit, co-pay and sub-limits. It must quote the exact sentence, and the quote must contain the number.
- **Query triage officer:** reads an insurer's query and decides: do we need to send documents, or just explain?
- **Claims desk reply writer:** writes the reply to the insurer, using only real facts from the case.

Insurer side (7 agents)

- **Identity:** compares patient details on the documents with the member record.
- **Authenticity:** explains warning signs like a missing stamp, wrong totals or duplicate bills.
- **Coverage:** finds and quotes the policy clauses that apply.
- **Calculation mapper:** maps bill lines to the policy's groups for the calculation engine.
- **Supervisor:** reads all the results and writes a short summary for the reviewer.
- **Query drafter:** writes the query to the hospital when something is missing.
- **Triage:** reads the hospital's reply and checks if it answers the query.

5.4 The flows

Flow	Steps in simple words
Hospital ClaimFlow	Load the case → is this a new build or a repair? → build the claim (or repair it) → estimate the payable amount → save the draft
Hospital QueryFlow	Load the query → decide: triage it, or draft a reply → do that job
Insurer VerificationFlow	Identity → authenticity → coverage → calculation mapping → supervisor → final outcome
Insurer QueryFlow	Draft a query to the hospital, or triage the hospital's reply

6. Who does what

Think of the system as an office with five kinds of workers. Each one has one job, and nobody does someone else's job.

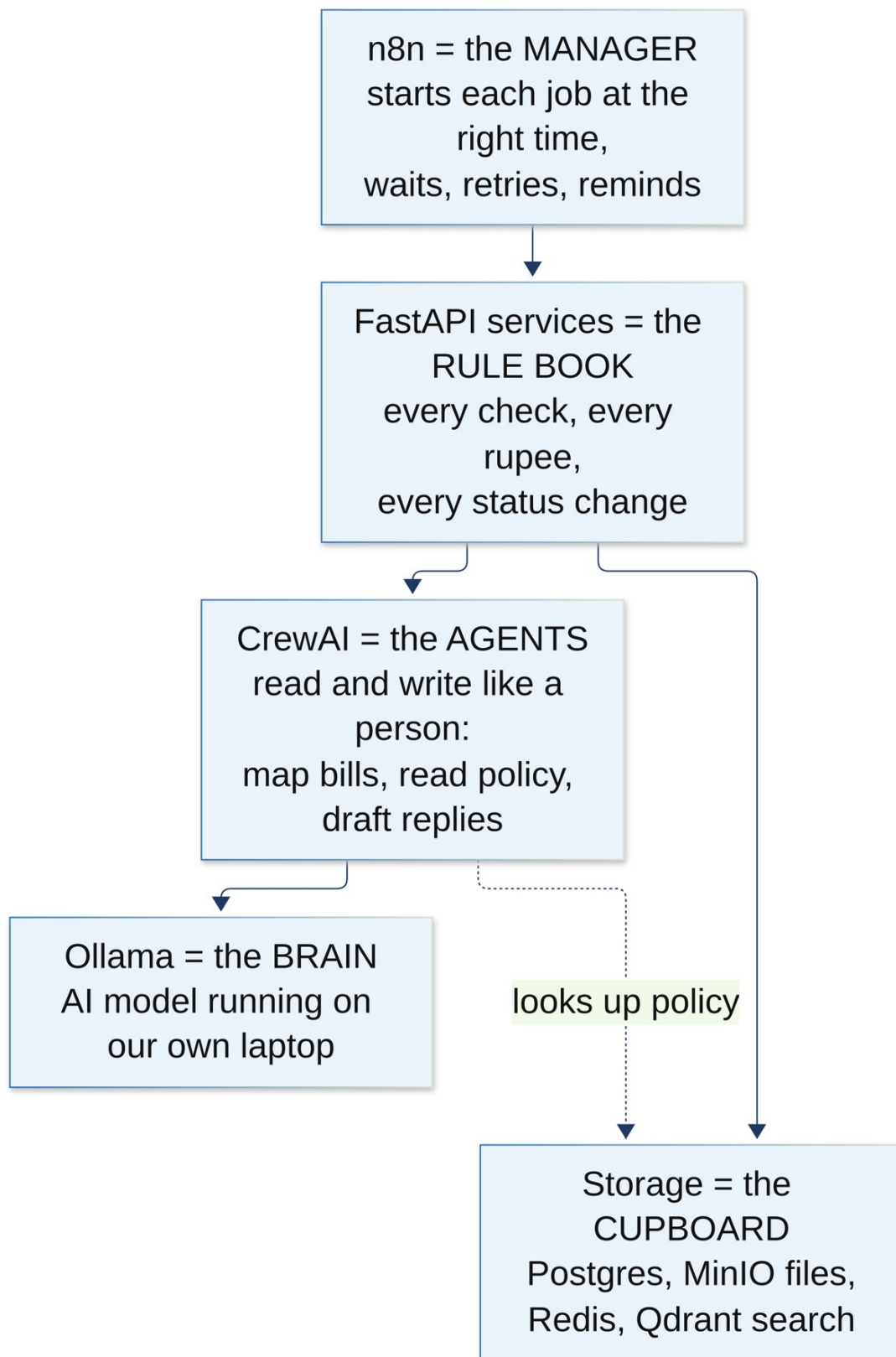


Figure 5: The five roles in the system

- **n8n is the manager.** It doesn't do the work. It starts each job at the right time, waits, retries, sends reminders.
- **CrewAI agents are the clerks who read and write.** They read bills and policies and write drafts.
- **The FastAPI services are the rule book.** Every check, every rupee, every change of status happens here.
- **Ollama is the brain** the agents use. It runs the AI model on our own laptop.
- **Storage is the cupboard:** databases, files, search index.

7. The tech stack: every tool and why

Here is every tool we used. For each one: what it is, what it does in our project, and why we chose it.

7.1 The AI part

CrewAI

What it is: A Python framework for building teams of AI agents.

What it does for us: Runs all our agents and flows on both sides, and checks their answers with guardrails.

Why we picked it: The guidelines need a multi-agent framework. CrewAI is simple, popular, written in Python like the rest of our code, and has Flows for clear step order.

Ollama (with gemma4 and nomic-embed-text)

What it is: A program that runs AI models on your own computer.

What it does for us: gemma4 is the model the agents think with. nomic-embed-text turns text into numbers so we can search the policy documents.

Why we picked it: Patient data stays on the laptop, it's free, and it works offline. No OpenAI key needed.

Qdrant + our RAG service

What it is: Qdrant is a database for searching by meaning. RAG means 'look it up first, then answer'.

What it does for us: When an agent needs the policy wording, the RAG service finds the right paragraphs so the agent can quote them.

Why we picked it: So agents quote real text instead of making things up. Hospital and insurer have separate collections, and each one has its own access token.

Presidio

What it is: A Microsoft tool that finds personal details in text.

What it does for us: Hides names, phone numbers and ID numbers before any text goes to the AI.

Why we picked it: Privacy. Even our local model only sees what it needs.

7.2 The workflow part

n8n

What it is: A workflow tool where you connect boxes (nodes) to automate a process.

What it does for us: Runs 13 hospital workflows and 19 insurer workflows: when a file is uploaded, when a claim arrives, every 5 minutes, every night, and so on.

Why we picked it: Claims are long processes with waiting and reminders. n8n is made for that, and you can see the whole process as a picture. It is open source and runs in Docker.

7.3 The backend part

Python 3.12 + FastAPI

What it is: Python is our main language. FastAPI is a framework for building web APIs.

What it does for us: All our services are FastAPI apps: hospital API, insurer API, crews, calculation engine, document pipeline, vision service, RAG service, TPA simulator.

Why we picked it: Fast, easy, and it checks every request automatically. It also creates API documentation by itself.

Pydantic

What it is: A library that checks data has the right shape.

What it does for us: Every request, every response and every agent answer is checked against a schema.

Why we picked it: Bad data is stopped at the door instead of causing trouble later.

SQLAlchemy + Alembic

What it is: SQLAlchemy talks to the database from Python. Alembic keeps track of database changes.

What it does for us: Stores cases, documents, claims, audit logs. Alembic upgrades the database step by step (the hospital is at step 0023).

Why we picked it: Anyone can build the same database from scratch with one command.

Calculation engine (our own code)

What it is: Plain Python that works out money using exact decimal numbers.

What it does for us: Applies room rent limits, sub-limits, co-pay, deductions and the sum insured to find the payable amount.

Why we picked it: Money must be exact and explainable. We never let AI calculate money.

7.4 The storage part

PostgreSQL 16

What it is: A reliable open-source database.

What it does for us: One database for the hospital, one for the insurer. Holds all records and the audit chain.

Why we picked it: Trusted, free, supports user roles so each service only gets the access it needs.

Redis 7

What it is: A very fast in-memory store.

What it does for us: Remembers which requests were already handled (so nothing runs twice), limits request rates, and carries live events to the screen.

Why we picked it: Very fast and simple for small, short-lived data.

MinIO

What it is: A file store that works like Amazon S3, but on our machine.

What it does for us: Stores all the uploaded scans and PDFs, encrypted.

Why we picked it: Same API as S3, so it could move to the cloud later. We build it from its source code because the official image was removed from Docker Hub.

7.5 The safety part

ClamAV

What it is: A free antivirus scanner.

What it does for us: Scans every uploaded file before anything opens it.

Why we picked it: Hospitals receive files from outside. A virus must never reach the system.

Keycloak

What it is: A login and user-roles server.

What it does for us: Handles login for desk staff, approvers and admins, and gives tokens to our services.

Why we picked it: Industry-standard login, so we don't write our own (which is risky).

HMAC signatures and JWT tokens

What it is: Ways to prove a message is genuine.

What it does for us: Every message between hospital and insurer is signed. Services use tokens to call each other.

Why we picked it: Nobody can fake or change a claim on the way.

7.6 The document part

Poppler, Tesseract and OpenCV

What it is: Poppler turns PDFs into images, Tesseract reads text from images (OCR), OpenCV works with images.

What it does for us: Read scanned bills, check photo quality, and look for the hospital stamp.

Why we picked it: Free, proven tools that run on the laptop.

7.7 The screen part

Next.js + React + TypeScript

What it is: A framework for building web pages.

What it does for us: The hospital desk screen and the insurer reviewer screen. It shows the checklist, the claim, the estimate card, queries and decisions.

Why we picked it: Modern, fast and widely used.

7.8 The 'make it run' part

Docker and Docker Compose

What it is: Docker runs software in boxes called containers. Compose starts many containers together.

What it does for us: Starts Postgres, Redis, MinIO, ClamAV, Keycloak, Qdrant and n8n with the right settings.

Why we picked it: It works the same on every computer. No manual install of each tool.

uv and Make

What it is: uv installs Python packages very fast. Make runs named commands.

What it does for us: uv sets up all our Python projects at once. Make gives short commands like `make demo`.

Why we picked it: One command runs the whole thing.

pytest, ruff, mypy, Playwright, Mermaid

What it is: Testing, code style, type checking, browser testing, drawing diagrams.

What it does for us: Hundreds of tests check everything, and Playwright clicks through the screens like a user. The diagrams are drawn from text.

Why we picked it: So we know it works, and we know when something breaks.

8. Triggers: what starts each workflow

A **trigger** is the thing that wakes up a workflow. We use four kinds:

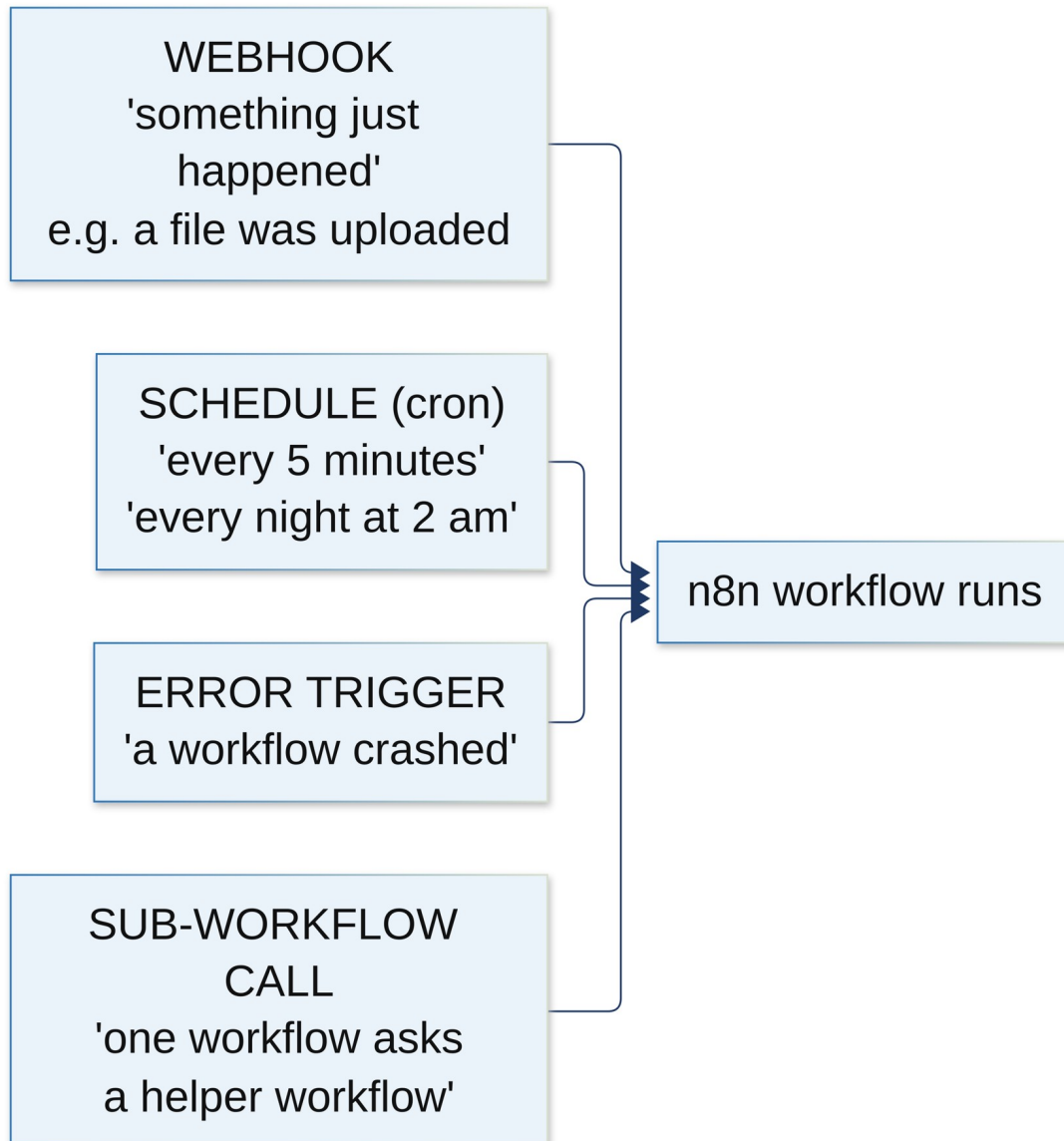


Figure 6: The four kinds of trigger we use in n8n

Trigger	Simple meaning	Example
Webhook	The API calls n8n and says 'this just happened'	A file was uploaded, a claim arrived, a query was sent
Schedule (cron)	A clock starts it again and again	Every 2 or 5 minutes, every hour, every night at 2 am
Error trigger	Runs when any other workflow crashes	Tell the admin and mark the job as failed
Sub-workflow call	One workflow asks a small helper workflow to do a common job	Get a login token, check for duplicates, call the agents

Every webhook is protected. It needs a secret (hospital) or a signature (insurer), so nobody else can start our workflows. The second PDF explains every workflow one by one, with screenshots.

9. How the decision is made

9.1 The decision gate

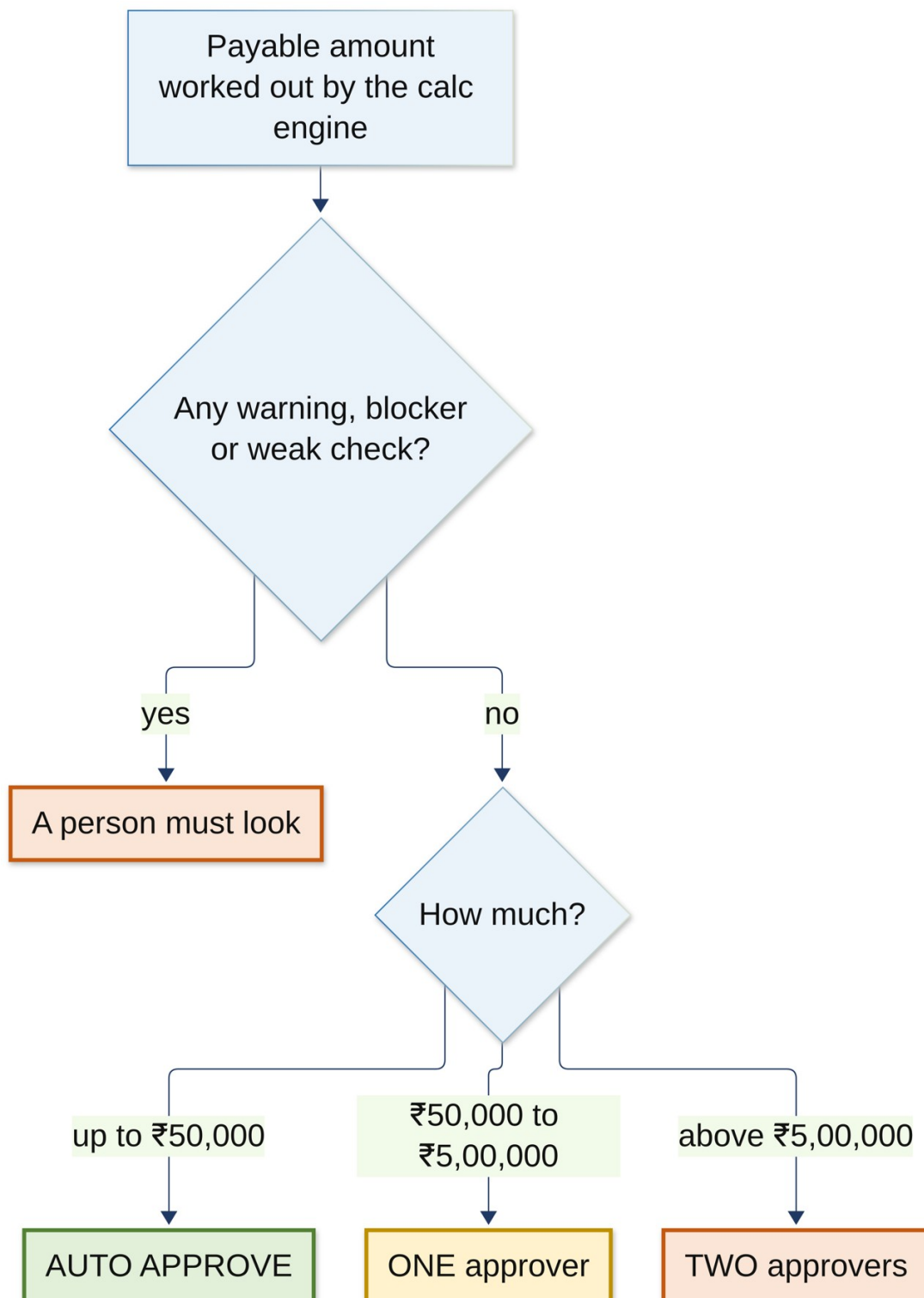


Figure 7: The decision gate

Even a small claim goes to a person if any check gave a warning, or if any step didn't work properly. The system can never auto-approve a doubtful claim. In our test of 300 claims, **wrongful automatic approvals were zero**.

9.2 The estimate: an example

Before the claim is sent, the hospital already knows roughly how much the insurer will pay. Here is how it works for a ₹55,800 bill on a ₹3 lakh policy with 10% co-pay:

Step	Amount (₹)	Why
Total bill	55,800	Room 4 days × 2,500 + operation 45,000 + attendant 800
Remove attendant charge	– 800	Not a medical cost, so not covered
Room rent check	0	Limit is 1% of 3 lakh = 3,000/day. 2,500 is under the limit
Eligible amount	55,000	
Co-pay 10%	– 5,500	The policy says 'A co-payment of 10% applies'
Estimated payout	49,500	The patient pays 6,300

The agent reads the policy and finds '10% co-pay' and '1% room rent', with the exact sentence as proof. The calculation engine does the maths. The estimate is only advice. It never blocks the claim.

10. The parameters: settings you can change

Parameters are the settings that control how the system behaves. Here are the important ones, in simple words.

10.1 Business settings (the rules)

Setting	Value	What it means
T_auto	₹50,000	Clean claims up to this amount can be approved by the system
T_four	₹5,00,000	Above this, two people must approve ('four eyes')
Room rent limit	1% of sum insured per day	Pay for the room only up to this. A costlier room reduces other charges in the same ratio
ICU limit	2% of sum insured per day	Same idea for the ICU
Co-pay	0%, 10% or 20%	The share the patient always pays. It depends on the product
Sub-limits	e.g. cataract ₹40,000	Fixed maximum for some treatments
Query rounds	3	After 3 rounds of questions, a senior reviewer takes over

Products in our data: HEALTH-BASIC (10% co-pay from age 61), HEALTH-PLUS-GOLD (no co-pay), SENIOR-SHIELD (20% co-pay, room 1.5%, ICU 3%).

10.2 AI and agent settings

Setting	What it does
CREW_LLM = ollama / rules	Hospital agents use the real model, or a rule-based stand-in with no model
INS_CREW_LLM = ollama / offline	The same switch for the insurer agents
INS_CREW_MODEL = gemma4:latest	Which AI model the agents use
RAG_EMBED_MODEL = nomic-embed-text	Which model turns text into numbers for search
guardrail retries = 1	If an agent's answer fails the check, it gets one more try with feedback
max_retry_limit = 0	If something crashes, tell our code right away instead of hiding it
Crew timeout = 100 s	n8n waits up to 100 seconds for an agent; if it's busy, it waits and tries up to 3 times

Setting	What it does
<code>CREWAI_TELEMETRY_OPT_OUT = true</code>	CrewAI sends no usage data out of the machine

10.3 Running settings

Setting	What it does
Ports	Hospital API 8100, hospital UI 3100, hospital crew 8010, insurer API 8600, insurer crew 8610, calc engine 8620, insurer UI 3600, RAG 8400, n8n 5688 and 5689, Ollama 11434
API retries	n8n retries a failed call after 2 s, 8 s and 30 s
Idempotency key	A unique ID on every request, so the same thing never happens twice even if a message is sent twice
Secrets	Passwords, signing keys and tokens. All created automatically by <code>make init-secrets</code> into the <code>.env</code> file, never put on GitHub
<code>RAG_SEED_ON_START</code>	Load the sample policy documents when the RAG service starts

10.4 Parameters inside the code

If the examiner asks about the code, these are the main parts:

Code	Parameters and meaning
<code>run_task(name, prompt, llm, model, guardrail, retries)</code>	Runs one agent job. name = which agent from <code>agents.yaml</code> , prompt = the input, llm = the model, guardrail = the checker function, retries = how many extra tries
<code>FlowDeps(llm, api, st, cancel, rag)</code>	What a flow needs: the model, the API client, the settings, a cancel signal, and the RAG search client
<code>agents.yaml</code> : role, goal, backstory	Describes each agent in words. CrewAI turns this into the agent's instructions
Guardrail returns (True, value) or (False, reason)	True means the answer is accepted. False sends the reason back to the agent to fix it
<code>@start, @listen, @router</code>	Flow markers: where the flow begins, which step runs after which, and where it splits into paths

11. Keeping data safe

- **Personal details are hidden** before any text goes to an AI model.
- **The AI runs locally**, so nothing goes to an outside company.
- **Every file is virus-scanned** first.
- **Files are encrypted** in storage.
- **Every message between the two sides is signed**, and duplicates are blocked.
- **Each service has its own password** with only the access it needs.
- **The audit log is chained with hashes**, so if anyone edits an old record, the check fails.
- **All the data in the project is made up** (synthetic). No real patient data is used.

12. How to run it

```
make demo-check      # are Docker, Ollama, models, ports ready?
make demo-offline    # quick run without the AI model (~1 min)
make demo            # full run with the real AI model
make demo-down       # stop everything

make crew-demo       # only the hospital agents
make ins-crew-demo   # only the insurer agents
make crew-plot       # draw the CrewAI flows as web pages
```

make demo does everything in order: checks your computer, starts the containers, sets up both databases, loads the policy documents into search, starts all services and both agent teams, imports the n8n workflows, runs claims from start to end, and prints a score card. If any step fails, it says which one and where the log is.

12.1 What we got

Check	Result
Full demo with the real model on a MacBook Air	Passed. One claim from upload to payment in 166 seconds
Offline demo	Passed. 35 seconds
Calculation vs an independent calculator (300 claims)	100% match
Wrong automatic approvals	0
Bill line mapping accuracy	100%
Finding the right policy text (recall at 5)	93%
Quoting exactly the right text (citation precision)	61%. Target is 90%. Still to improve

13. What is still left, and what comes next

- **Better quoting:** citation precision is 61%, the target is 90%. Fix it with smarter splitting of documents and better ranking.
- **Measure the real benefit:** how much time is saved compared to doing it by hand, how many fewer queries.
- **Make the insurer API use the CrewAI flow** for all verification, not just in the demo.
- **Move document reading and validation into CrewAI agents** too.
- **Faster AI:** on a laptop CPU, a reply can take minutes. A GPU would help.
- **Test on real (de-identified) scans** from a hospital.
- **Later:** pre-authorisation requests, connecting to the government's National Health Claims Exchange (NHCX), and Hindi and other Indian languages.

14. Questions the examiner may ask

Q: Why not let the AI decide the claim?

Because AI can make mistakes and can't always explain itself. Money and rules need to be exact. So the AI reads and writes, the code decides, and people approve the big ones.

Q: Why CrewAI and not LangChain or AutoGen?

CrewAI is made exactly for role-based agent teams, it has Flows for a clear step order and guardrails for checking answers, and it is simple to read. The guidelines asked for a multi-agent framework, and CrewAI fits our design best.

Q: Why n8n if you already have CrewAI?

They do different jobs. CrewAI handles the thinking inside one step. n8n handles the business process around it: when to start, how long to wait, reminders, retries, alerts. A claim can take days, and n8n is made for that.

Q: What if the AI model is down?

The step is marked 'degraded', the claim continues, and it goes to a person. Nothing is lost. We also have an offline mode with no model at all.

Q: How do you stop the AI from making things up?

Guardrails. The agent must quote the real text, and the code checks the quote really exists and contains the number. If not, the agent tries again, and if it still fails, the code's answer is used.

Q: How do you protect patient data?

The model runs locally, personal details are hidden before the model sees anything, files are encrypted, and every message is signed.

Q: Why two separate systems?

Because a hospital and an insurer are different companies. They can't share a database. Our design copies real life.

Q: What is the one thing you're most proud of?

One command builds and runs the whole system end to end on a laptop, and in 300 test claims it never wrongly auto-approved a claim.